

Names, Bindings, Scope, and Lifetime

Liting Zhang

University of West Florida

Overview

- How programs connect names to values and locations
- How scope and lifetime control what is visible and when
- Static scope and a thought experiment on dynamic scope
- Storage lifetime: static, stack, heap, closures
- Referential transparency and side effects
- Bugs from shadowing and hoisting
- How TypeScript let and const help compared to var

Names and what they refer to

When we write programs, we work with names and not raw addresses.

- A name is a word in the source program that stands for something else. Examples:
 - ▶ Variable names such as `count`, `total`, `studentName`
 - ▶ Function names such as `print`, `sort`, `push`
 - ▶ Type names such as `int`, `double`, `string`
- A value is the data currently stored. Examples:
 - ▶ The integer `42` stored somewhere
 - ▶ The string `"hello"` stored somewhere
- A memory location is a place in memory, such as an address or offset, that can hold a value.

Programmers mostly see names, the machine mostly sees addresses. A language needs rules to connect these two worlds.

Names: design issues in languages

When designing a language, there are many choices about how names work.

Questions language designers must answer:

- What characters are allowed in names
 - ▶ letters, digits, underscore, dollar sign
 - ▶ non-ASCII characters or Unicode letters
- Are names case sensitive
 - ▶ count, Count, and COUNT the same or different
- How long can names be
 - ▶ some old languages had short limits (for example 6 or 8 characters)
 - ▶ most modern languages allow long descriptive names
- Which names are reserved words
 - ▶ keywords such as if, while, class cannot be used as identifiers
- Which kinds of things can be named
 - ▶ variables, constants, functions, classes, modules, types

These choices affect readability, tool support, and how easy it is to make mistakes.

Name forms in C, Python, and TypeScript

Many languages have similar rules, but there are some differences.

```
1 // C++
2 int total_sum = 0;
3 double x2 = 3.14;
4 // int class = 1;    // not allowed: class is a keyword in C++
1 # Python
2 student_name = "Ana"
3 _max_count = 10 # leading underscore often means "internal use"
1 // TypeScript / JavaScript
2 let totalSum = 0;
3 const $cacheKey = "users";    // £ is allowed
4 // let if = 1; // not allowed: if is a keyword
```

- All three are case sensitive: Name, name, and NAME are different.
- C and TypeScript typically use camelCase for variables, Python often uses snake_case.
- Underscore and dollar sign are allowed in TypeScript and many other languages.
- Keywords are reserved and cannot be reused as variable names.

What kinds of things get names

Names are not only for variables. A typical program has many kinds of named entities.

Examples of named things:

- Variables
 - ▶ local variables inside a block or function
 - ▶ parameters to functions
 - ▶ global or module level variables
- Functions and methods
 - ▶ print, sort, addStudent, computeArea
- Types and classes
 - ▶ struct Node in C
 - ▶ class Student in TypeScript
- Modules, packages, or namespaces
 - ▶ Python modules, TypeScript modules or files

The symbol table in a compiler needs to record not only the name string, but also what kind of thing each name refers to.

Bindings and environments

A binding connects parts of the program description to each other.

Examples of bindings:

- A name bound to a memory location
 - ▶ Example: in C, the name `x` is placed at some offset in a stack frame
- A name bound to a type
 - ▶ Example: the compiler records that `x` has type `int`
- A function name bound to a piece of code
 - ▶ Example: the name `max` refers to the code that computes a maximum

The environment is the collection of all bindings that are currently in effect during execution.

- At a given point in the program, each visible name has some meaning: where it is stored, what type it has, what value it currently holds.

Binding Types

A single declaration can create several different bindings.

- Declaration versus assignment versus initialization
 - ▶ Declaration introduces a name and its type
 - ★ C: `int x;`
 - ▶ Initialization gives a first value at the same time
 - ★ C: `int x = 0;`
 - ▶ Assignment changes the value later
 - ★ C: `x = x + 1;`
- Different kinds of information can be bound
 - ▶ Type binding: associate a name with a type
 - ★ x has type `int`
 - ▶ Storage binding: associate a name with a location
 - ★ x is at some offset in the current stack frame
 - ▶ Value binding: associate a location with a current value
 - ★ the location for x currently holds 42
- Early versus late binding
 - ▶ Some bindings are fixed at compile time, such as the type of x.
 - ▶ Others are decided at run time, such as which method a virtual call uses in an object oriented language.

When do bindings happen

A binding does not always happen at the same time.

Typical binding times:

- Language design time
 - ▶ designers decide which keywords exist and what the primitive types are
- Compile time
 - ▶ the compiler decides that `x` has type `int`
 - ▶ the compiler decides where in the stack frame `x` will live
- Link and load time
 - ▶ the linker decides the address of global variables and functions
- Run time
 - ▶ parameter values are bound to parameters when a function is called
 - ▶ heap objects are given locations when allocated

Earlier bindings allow more static checking and optimization.

Later bindings allow more flexibility, but some errors only show up when you run the program.

Bindings in C

```
1  #include <stdio.h>
2  // name -> static storage location
3  int global_counter = 0;
4  int increment(int x) {
5      // x is bound to argument on each call
6      int local = x + 1;    // local has stack storage
7      global_counter++;    // uses global binding
8      return local;
9  }
10 int main(void) {
11     int n = 10;
12     int result = increment(n);
13     printf("%d %d\n", result, global_counter);
14 }
```

- x and local are created and destroyed each call
- global_counter is created once and lives for the whole program

Symbol tables in a compiler

- Internally, a compiler or interpreter needs a data structure to track bindings.
- This structure is usually called a symbol table.
- For each name that has been declared, the symbol table may store:
 - ▶ the string that spells the name
 - ▶ what kind of thing it is (variable, function, type, parameter, etc.)
 - ▶ its type
 - ▶ which scope it belongs to (block, function, module)
 - ▶ where it is stored (stack offset, global address, or heap information)
 - ▶ perhaps extra flags for debugging or optimization
- There is often not just one table but a family of tables, one per scope, connected in a tree-like structure.

Toy symbol table operations

```
1  open_scope()
2      create a new table with parent = current scope
3      set current scope to this new table
4
5  insert(name, info)
6      add name -> info to current scope
7      if name already exists in current scope, report an error
8
9  lookup(name)
10     search current scope for name
11     if not found, search parent scope
12     repeat until found or no more parents
13     if not found at all, report an error
14
15  close_scope()
16     set current scope to its parent
```

The static semantics of a language (such as "name already declared" or "name not declared") follows from simple rules on these symbol tables.

Symbol tables and nested scopes

```
1  int x = 1;           // global scope
2  void f() {
3      int y = 2;       // function scope
4      {
5          int z = 3;    // inner block scope
6          int x = 4;    // shadows global x
7      }
8  }
```

We can imagine one symbol table per scope:

- Global table
 - ▶ x, kind = variable, scope = global
 - ▶ f, kind = function, scope = global
- Function table for f, parent = global
 - ▶ y, kind = variable, scope = f
- Inner block table, parent = f
 - ▶ z, kind = variable, scope = inner block
 - ▶ x, kind = variable, scope = inner block

lookup(x) in the inner block first finds x in the inner block table and does not see the global x. lookup(y) in the inner block climbs to the function table and finds y there.

What scope means

Scope answers the question: where in the program text does a binding apply?

- If a variable is declared in one block, can you use it in a different block?
- If two variables in different places have the same spelling, are they the same variable or different ones?

Two main models:

- Static scope (also called lexical scope)
 - ▶ scope depends on the structure of the program text
- Dynamic scope
 - ▶ scope depends on which functions called which at run time

Most modern languages (TypeScript, C, Python, Java, etc.) use static scope.

Static scope example in TypeScript

```
1  let x = 1;
2  function outer() {
3      let y = 2;
4      function inner() {
5          let x = 3;
6          console.log("x:", x, "y:", y);
7      }
8      inner();
9  }
10 outer();
```

- The name `x` inside `inner` refers to the `x` declared inside `inner` (the closest declaration in the surrounding text).
- The name `y` inside `inner` refers to the `y` declared in `outer`, because `inner` is textually inside `outer`.
- The global `x` is hidden inside `inner` by the inner `x`.

The meaning of `inner` is determined completely by the surrounding code, not by who calls `outer` or `inner`.

Block scope and shadowing in C

```
1  #include <stdio.h>
2  int main(void) {
3      int x = 10;
4      printf("outer x = %d\n", x);
5      {
6          int x = 20;  // shadows outer x in this block
7          printf("inner x = %d\n", x);
8      }
9      printf("outer x again = %d\n", x);
10 }
```

- The outer declaration of `x` is visible from its declaration down to the end of `main`, except where it is shadowed.
- Inside the inner block, there is a new declaration of `x`. This `x` hides the outer `x` inside the block.
- After the inner block ends, the inner `x` goes away, and the outer `x` is visible again.

Dynamic scope thought experiment

Many mainstream languages do not use dynamic scope, but it is useful to understand what it would do.

```
1  var x = 1;
2  function f() {
3      print(x);
4  }
5  function g() {
6      var x = 2;
7      f();
8  }
9  g();
```

- Static scope:
 - ▶ f looks at the place it was defined.
 - ▶ It sees the global x and prints 1.
- Dynamic scope:
 - ▶ f looks at who called it.
 - ▶ It sees x in g and would print 2.

Under dynamic scope, changing who calls f might silently change what x it sees.

Static scope in practice

Reasons static scope is preferred in most modern languages:

- You can read a function and determine which variable each name refers to by following the source code structure.
- Tools like editors and IDEs can
 - ▶ jump from a use of a name to its definition
 - ▶ rename a variable safely
- Refactoring (such as moving code around or changing callers) does not suddenly change which x a function sees.

Dynamic scope can be useful in narrow situations but tends to make larger programs hard to reason about.

Python LEGB rule

Python uses lexical scope but describes it using the LEGB order:

- L: local scope inside the current function
- E: enclosing function scopes (if there are nested functions)
- G: global scope (names defined at module level)
- B: builtins (names that Python provides automatically)

When Python sees a name, it searches in this order until it finds a binding. Python also provides two keywords that change how assignments behave:

- global: assign to a name in the module scope
- nonlocal: assign to a name in an enclosing (but not global) function scope

Python LEGB rule in a small example

```
1 | x = "global"
2 | def outer():
3 |     x = "outer"
4 |     def inner():
5 |         print(x)
6 |     inner()
7 | outer()
```

Applying LEGB:

- inner looks for x.
- L (local to inner): no x is assigned here, so no match.
- E (enclosing functions): outer has x = "outer", so that is used.
- G (global): x = "global" is ignored, because a match was already found.
- B (builtins): not reached.

The program prints outer, not global, because the name x in inner is resolved to the nearest enclosing function scope.

Python enclosing scope and nonlocal

```
1 def make_counter():
2     count = 0
3     def inc():
4         nonlocal count
5         count += 1
6         return count
7     return inc
8 c1 = make_counter()
9 print(c1()) # 1
10 print(c1()) # 2
11 c2 = make_counter()
12 print(c2()) # 1 (separate counter)
```

- count is created when make_counter is called.
- inc is a function that lives inside make_counter and refers to count.
- nonlocal count means: update the count that lives in the nearest enclosing function scope.
- Each call to make_counter creates a new count and a new inc that refers to that count, so c1 and c2 are independent counters.

Python global example

```
1 counter = 0  # module level (global) name
2
3 def bump():
4     global counter
5     counter += 1
6
7 bump()
8 bump()
9 print(counter)  # 2
```

- The name `counter` is first defined at module level with value 0.
- The function `bump` uses the `global` keyword to say that assignment to `counter` should change the module level `counter`.
- Each call to `bump` increases the shared global `counter`.

This pattern is sometimes convenient but tends to spread state across the program, which can make debugging harder.

Scope versus lifetime

It is easy to mix up scope and lifetime, but they are different ideas.

- Scope is about the region in the code where a name refers to a particular binding.
- Lifetime is about the period during program execution when a piece of storage actually exists.

Examples:

- A variable can be out of scope but still alive if it is captured in a closure and the closure is stored somewhere else.
- A variable can be in scope but not yet initialized, which can lead to bugs if you use it too early.

Storage lifetime

Most languages distinguish three main lifetimes:

- Static or global lifetime
 - ▶ storage exists from program start until program end
 - ▶ used for global variables and sometimes for static locals
- Stack or automatic lifetime
 - ▶ storage is allocated when entering a function or block
 - ▶ storage is reclaimed automatically when leaving that function or block
- Heap or dynamic lifetime
 - ▶ storage is requested explicitly (for example with malloc or new)
 - ▶ storage is freed explicitly or by a garbage collector

Languages differ in which combinations they support and how they manage heap storage.

C storage duration example

```
1  #include <stdlib.h>
2  int global_var = 0;           // static duration
3  void foo(void) {
4      static int cached = 0;    // static duration, but block scope
5      int local = 1;            // automatic (stack) duration
6      int *heap_ptr = malloc(sizeof(int)); // heap allocation
7      *heap_ptr = local + cached;
8      cached++;
9      free(heap_ptr);
10 }
```

- `global_var`: static lifetime. It is allocated before `main` starts and exists for the entire run of the program.
- `cached`: static lifetime, but cannot be seen outside `foo`.
- `local`: automatic stack lifetime. lives only during a call to `foo`.
- `heap_ptr` lives on the stack in `foo`, but the integer it points to lives on the heap between `malloc` and `free`.

Check your understanding: lifetime

```
1  #include <stdlib.h>
2
3  int global_flag = 0;
4
5  int *make_ptr(void) {
6      static int stored = 0;
7      int local = 1;
8      int *heap_val = malloc(sizeof(int));
9      *heap_val = 2;
10     return heap_val;
11 }
```

For each of the following, which lifetime does it have?

- global_flag: static lifetime
- stored: static lifetime, but block scope
- local: stack lifetime
- the integer pointed to by heap_val: heap lifetime

Lifetime bugs in C

```
1  #include <stdio.h>
2  int *bad_pointer(void) {
3      int local = 42;    // stack storage
4      return &local;    // returns address of local
5  }
6  void leak(void) {
7      int *p = malloc(sizeof(int)); // heap
8      *p = 10; /* forgot to call free(p); */
9  }
10 int main(void) {
11     int *q = bad_pointer();
12     printf("%d\n", *q); // undefined behavior: q points to dead stack memory
13     leak();             // memory is never freed
14 }
```

- Returning the address of a local variable creates a dangling pointer. The stack slot for local becomes invalid when bad_pointer returns.
- Forgetting to free heap memory in leak causes a memory leak, where allocated storage is never reclaimed.
- Lifetime errors are a major source of bugs in low level languages with manual memory management.

Stack and heap mental model

Stack storage:

- very fast to allocate and free
- last in, first out discipline: the most recent call is the first to return
- good for temporary variables and parameters
- size is usually limited

Heap storage:

- can be allocated and freed in any order
- can hold objects whose lifetime does not match a single function call
- often managed by the programmer (C) or by a garbage collector (Python, TypeScript)
- can lead to memory leaks or fragmentation if not managed carefully

Closures and extended lifetime in TypeScript

```
1 function makeAdder(n: number) {  
2     return function (x: number) {  
3         return x + n;  
4     };  
5 }  
6 const add10 = makeAdder(10);  
7 console.log(add10(5));    // 15  
8 console.log(add10(20));   // 30
```

- When `makeAdder` runs, it creates a parameter `n` and then returns an inner function that uses `n`.
- That inner function is a closure: it carries an environment that remembers the value of `n`.
- Even after `makeAdder` has returned, `n` is kept alive because `add10` still uses it.
- The lifetime of `n` is therefore tied to the lifetime of the closure, not to the stack frame of `makeAdder`.

Lifetime in C versus garbage collected languages

C and languages like Python or TypeScript handle storage lifetime differently.

- In C

- ▶ Static and stack lifetimes are managed by the compiler and the call stack.
- ▶ Heap lifetimes are managed by the programmer using malloc and free.
- ▶ Bugs such as leaks, use after free, and double free can happen if frees are not matched correctly to allocations.

- In Python or TypeScript

- ▶ Heap objects are allocated automatically when needed.
- ▶ A garbage collector reclaims objects when they are no longer reachable from any variables or closures.
- ▶ Program code can still keep references around longer than needed and effectively "leak" memory, but the collector prevents classic use after free errors.

Closures in garbage collected languages naturally extend the lifetime of captured values. In C, similar behavior must be programmed using heap allocation and careful manual lifetime management.

Expressions

When we run a piece of code, two different things can happen:

- it produces a value that can be used in a larger expression
- it may also change something else in the program or outside world

Expression:

- an expression is code that evaluates to a value
- examples:
 - ▶ `3 + 4` evaluates to 7
 - ▶ `square(5)` evaluates to 25
 - ▶ `"hi".upper()` evaluates to "HI"
- when we only care about the value, we can think in a math-like way

Side effects

Side effect:

- a side effect is any change that happens in addition to producing a value
- typical side effects:
 - ▶ updating a variable or a field
 - ▶ printing to the screen or writing to a file
 - ▶ modifying a list, dictionary, or global state

A useful habit when reading code:

- first ask: what value does this expression compute
- then ask: what else does this code change while computing that value

Referential transparency

Referential transparency is a property of expressions and functions.

- An expression is referentially transparent if replacing that expression with its value does not change the observable behavior of the program.
- A function is referentially transparent if
 - ▶ its result depends only on its arguments
 - ▶ it does not cause visible side effects when called

Examples:

- $3 + 4$ is referentially transparent: it always evaluates to 7 and does not change anything else.
- A function that reads and updates a global counter is not referentially transparent, because replacing the call with its result would change the value of the counter.

Referential transparency is a key idea in functional programming, because it supports algebra like reasoning and easier testing.

Pure and impure functions in Python

```
1  # pure function
2  def square(x):
3      return x * x
4  # impure function with side effect
5  acc = 0
6  def add_to_acc(x):
7      global acc
8      acc += x
9      return acc
```

- `square(x)` depends only on `x` and does not change any external state. Calling `square(3)` is always 9, no matter when you call it.
- `add_to_acc(x)` reads and updates the global variable `acc`. The result depends on how many times you have called it before.
- If you see the call `add_to_acc(3)` in a large program, you cannot know its result without knowing the entire history of `acc`.

Side effects and hidden state in TypeScript

```
1 let cache: number[] = [];  
2 function computeAndCache(x: number): number {  
3     const result = x * x;  
4     // side effect on outer binding cache  
5     cache.push(result);  
6     return result;  
7 }  
8 computeAndCache(3);  
9 computeAndCache(3);
```

- The function `computeAndCache` both returns a value and modifies the array `cache` defined outside the function.
- The second call to `computeAndCache(3)` behaves differently from the first if other code looks at `cache`.
- This style is sometimes needed, but it makes reasoning about the program harder than if we only had pure functions.

Pure computation versus hidden state

```
1 function square(x: number): number {  
2     return x * x;  
3 }  
4 let cache2: number[] = [];  
5 function computeAndCache(x: number): number {  
6     const result = x * x;  
7     cache2.push(result); // side effect  
8     return result;  
9 }
```

- square computes x times x and returns the result.
 - ▶ Calling `square(3)` is always 9.
 - ▶ Nothing else in the program changes.
- `computeAndCache` also computes x times x , but it modifies `cache2`.
 - ▶ The state of `cache2` depends on how many times and in what order `computeAndCache` has been called.
 - ▶ Code elsewhere that reads `cache2` will see different results depending on this history.

When reading code, it is useful to separate what value a function returns from what hidden state it might change while computing that value.

Shadowing pitfall in C

```
1 int total = 0;
2 void add(int values[], int n) {
3     for (int i = 0; i < n; i++) {
4         int total = values[i]; // shadows global total
5         //code that was intended to update the global total
6     }
7 }
```

- The programmer probably meant to write `total += values[i];` and to refer to the global total.
- Instead, a new local variable `total` is declared inside the loop. This local `total` hides the global `total` inside the loop.
- The code compiles and runs, but the global `total` never changes.
- This kind of shadowing bug appears in many languages, not just in C.

Shadowing in TypeScript and Python

```
1 // TypeScript
2 let total = 0;
3 function add(values: number[]): void {
4     for (let i = 0; i < values.length; i++) {
5         let total = values[i];    // shadows outer total
6         // intended: total += values[i];
7     }
8 }

1 #python
2 total = 0
3 def add(values):
4     for v in values:
5         total = v    # rebinds global total without using it
6         # intended: total += v
```

- The name `total` is reused in a way that hides or replaces the original meaning.
- The code compiles or runs, but it does not update the intended accumulation variable.
- Many teams use linters that warn when a local variable shadows a

Hoisting with var in JavaScript or TypeScript

```
1 function demo() {  
2     console.log(msg); // prints undefined, not a  
    ↪ ReferenceError  
3     var msg = "hello";  
4 }
```

The actual behavior is easier to see if we rewrite how the engine thinks:

```
1 function demo() {  
2     var msg;           // declaration is hoisted  
3     console.log(msg);  // msg exists but is undefined  
4     msg = "hello";  
5 }
```

The key points:

- Declarations with var are moved (hoisted) to the top of their function.
- The variable exists from the start of the function but has value undefined until the assignment line.
- This can hide bugs because reading msg before the assignment does not cause an immediate error.

Summary of var, let, and const

Common differences in JavaScript and TypeScript:

keyword	scope	before declaration	reassignment
var	function scope	hoisted, value undefined	allowed
let	block scope	temporal dead zone	allowed
const	block scope	temporal dead zone	not allowed

Notes:

- var creates one binding per function, visible throughout the function body and usable before its line with value undefined.
- let and const create bindings that are limited to the nearest block, such as a loop body or an if body.
- let and const names exist from the start of the block but cannot be used until the declaration line is executed, which is the temporal dead zone.
- const prevents reassignment of the variable name after initialization.

let and const: scope and temporal dead zone

```
1 function demoLetConst() {  
2   console.log(msg); // ReferenceError (temporal dead zone)  
3   let msg = "hello";  
4  
5   const greeting = "hi";  
6   console.log(greeting); // ok  
7   // greeting = "hey"; // error: cannot reassign a const  
8 }
```

- let and const are block scoped: the names only exist inside the nearest block, such as this function body.
- From the start of the block until the let or const line, the name is in the temporal dead zone and cannot be used. Reading msg before its let declaration causes a run time error.
- Variables declared with let can be reassigned after they are initialized.
- Variables declared with const cannot be reassigned to a new value. The name greeting will always refer to the same value after initialization.

Block scope in loops: let

```
1 function loopLet() {  
2   for (let i = 0; i < 3; i++) {  
3     setTimeout(() => console.log("let:", i), 0);  
4   }  
5 }  
6
```

- In loopLet, the declaration `let i` creates a new binding of `i` for each iteration of the loop body.
- Each arrow function captures the `i` from its own iteration, so the outputs are `let: 0`, `let: 1`, `let: 2`.

var in loops: classic closure bug

```
1 function loopVar() {  
2   for (var i = 0; i < 3; i++) {  
3     setTimeout(() => console.log(i), 0);  
4   }  
5 }  
6  
7 // prints: 3, 3, 3
```

- With var, the variable i is scoped to the whole function loopVar, not to the body of the for loop.
- The arrow function does not capture the value of i at each iteration; it captures a reference to the single variable i.
- By the time the timeouts run, the loop has finished and i has value 3, so each callback prints 3.
- Using let instead of var in the loop header creates a new binding for i at each iteration, which avoids this bug.